Lecture 23 : Project Estimation Techniques (Contd.)

Introduction

The landscape of software development has dramatically transformed since the early 1980s, when the original COCOMO (Constructive Cost Model) was first introduced. Modern software projects are not only significantly larger in scale but also heavily rely on the reuse of existing components, component-based development, and service-oriented architectures. Furthermore, the prevalence of graphical user interfaces (GUIs) and web-based applications has introduced new complexities, with GUI development often consuming a substantial portion of project effort—sometimes as much as the core functionality itself.

This lecture introduces COCOMO II, a crucial evolution of the original COCOMO model, designed to address these contemporary challenges. Proposed by Barry Boehm in 1995, COCOMO II provides a more refined and accurate approach to estimating software project costs and schedules, adapting to the life cycle practices and technological advancements of the 1990s and 2000s. Understanding COCOMO II is essential for project managers and developers to accurately forecast resources, manage expectations, and ensure the successful delivery of complex, modern software systems.

Core Concepts

Term	Definition	Significance
`COCOMO II`	An updated version of the Constructive Cost Model, designed to estimate software project costs and schedules in modern development environments.	It addresses the complexities of contemporary software, including reuse, GUIs, and new development paradigms, offering more accurate estimations than its predecessor.
`Application Points`	A size metric used in the Application Composition Model, calculated by counting screens, reports, and modules, weighted	Provides a way to estimate effort for prototyping projects and those with extensive reuse, especially for GUI-centric applications, by

Term	Definition	Significance
	by complexity (simple, medium, difficult).	focusing on user-facing components.
`Scale Drivers`	Five factors (Precedentedness, Development Flexibility, Architecture/Risk Resolution, Team Cohesion, Process Maturity) that determine the exponent in the COCOMO II effort equation.	They replace the development modes of COCOMO 81 and significantly influence a project's duration and cost by reflecting systemic project characteristics.
`Effort Multipliers`	Factors that adjust the estimated effort based on specific project attributes like product reliability, platform difficulty, personnel capability, and schedule pressure.	These 17 multipliers (in the Post-Architecture model) refine the effort estimate by accounting for various technical, personnel, and environmental influences.
`SLOC (Source Lines of Code)`	A size metric used in COCOMO II, where a single logical line of code (e.g., an if-then-else statement) is counted as one unit, regardless of physical lines.	It offers a more consistent and modern approach to measuring software size compared to DSI (Delivered Source Instructions), which could count multiple physical lines for a single logical instruction.
`Reuse Model`	A sub-model within COCOMO II specifically designed to estimate the effort involved in integrating reusable components or automatically generated code.	Crucial for modern development, as it accounts for the cost of assessing, selecting, assimilating, and potentially adapting existing code, whether black-box or white-box.
`Post-Architecture Model`	The most detailed COCOMO II sub-model, used once the system architecture is designed and more information	Provides the most accurate cost estimates by incorporating a comprehensive set of factors

Term	Definition	Significance
	is available, employing 17 effort multipliers.	after significant design decisions have been made.

Detailed Analysis

1. Evolution and Objectives of COCOMO II

The original COCOMO model, developed in the early 1980s, became less suitable for the evolving software development landscape. Key changes necessitated an update:
* **Paradigm Shifts**: The rise of component-based software development, service-oriented architectures (SOA), and new life cycle models like RAD, Scrum, and Extreme Programming.
* **Increased Size and Reuse**: Modern projects are larger and extensively reuse existing software components, requiring specific estimation for integration effort.
* **Graphical User Interfaces (GUIs)**: Unlike the 1980s where GUIs were rare, present-day software is highly interactive, with GUI development often consuming up to 50% of the total effort.
* **New Techniques**: The emergence of reverse engineering, reengineering, and automated tools (CASE tools) also needed to be factored into cost estimation.

Boehm proposed COCOMO II in 1995 to address these changes. Its main objectives are:
* To develop a software cost and schedule estimation model tuned to the life cycle practices of the 1990s and 2000s.
* To develop a software cost database and tool support capabilities for continuous model improvement.

COCOMO II offers three (now commonly considered four) models to provide increasingly accurate cost estimations at different phases of a project.

2. COCOMO II Models

COCOMO II incorporates a range of sub-models, each applicable at different stages of the software development lifecycle, producing increasingly accurate estimates as more project information becomes available. The four sub-models are:

- **Application Composition Model**:
- **Purpose**: Used for prototyping projects and those with extensive reuse, where software is composed from existing parts.
- **Basis**: Relies on standard estimates of developer productivity in terms of "application points" or "object points" per month.

- **Considerations**: Takes into account the use of CASE tools.
- **Early Design Model**:
- **Purpose**: Applied when requirements are available, but the detailed design has not yet started.
- **Basis**: Used for initial effort estimation based on system requirements and design options, often using function points.
- **Reuse Model**:
- **Purpose**: Computes the effort required to integrate reusable components, whether they are existing code, databases, or automatically generated code.
- **Considerations**: Distinguishes between "black-box" reuse (code used without modification) and "white-box" reuse (code that needs adaptation).
- **Post-Architecture Model**:
- **Purpose**: Used once the system architecture has been designed and more detailed information about the system is available.
- **Basis**: Provides the most refined estimates, using a larger set of associated multipliers compared to the Early Design Model.

3. Size Metrics: SLOC vs. DSI

COCOMO 81 used "Delivered Source Instructions" (DSI) as its size metric. COCOMO II, however, uses "Source Lines of Code" (SLOC).
* **DSI**: Could count several physical lines for a single logical instruction.
* **SLOC**: A single logical line of code, such as an "if-then-else" statement, is counted as one SLOC, even if it spans multiple physical lines. This provides a more consistent measure of logical complexity.

4. Application Composition Model in Detail

This model is particularly suitable for software built around Graphical User Interfaces (GUIs) and modern GUI-builder tools. It uses object points (or application points) as its size metric, which are an extension of function points.

- **Calculation of Application Points (NAP)**:
- NAP is a weighted estimate of three things:
- **Number of separate screens displayed**:
- Simple screens: 1 object point
- Moderately complex screens: 2 object points

- Very complex screens: 3 object points
- **Number of reports produced**:
- Simple reports: 2 object points
- Moderately complex reports: 5 object points
- Difficult reports: 8 object points
- **Number of modules**: Modules developed in languages like Java or C++ to supplement database programming code. Each module counts as 10 object points.
- **Effort Formula**:

```
PM = (NAP * (1 - %reuse / 100)) / PROD
```

Where:
* `PM` = Effort in person-months
* `NAP` = Number of application points
* `%reuse` = Percentage of code reused
* `PROD` = Productivity (application points per month), which varies based on developer experience and ICASE maturity/capability. For example, a very low developer experience and ICASE maturity might yield a PROD of 4, while very high could be 50.

5. Core Effort Equation and Scale Drivers

The core COCOMO II effort equation is:

```
pm = A * (size)^sf * (em1) * (em2) * (em3) ...
```

Where:
* `pm` = Effort in person-months
* `A` = A constant (typically 2.94)
* `size` = Number of thousands of lines of code (KSLOC)
* `sf` = Scale factor (exponent)
* `em` = Effort multipliers

Scale Drivers: These five factors replace the development modes of COCOMO 81 and determine the exponent (`sf`) in the effort equation. They are particularly sensitive to system size.

- **Precedentedness (PREC)**: Degree to which past examples can be consulted (e.g., similar projects handled before).

- **Development Flexibility (FLEX)**: Degree of flexibility in implementing the project (e.g., strict requirements vs. adaptable goals).
- **Architecture/Risk Resolution (RESL)**: Degree of uncertainty about requirements.
- **Team Cohesion (TEAM)**: Level of interaction and cooperation among team members.
- **Process Maturity (PMAT)**: Derived from SEI CMM/CMMI model, reflecting the maturity of the development process.

The `sf` (scale factor) is calculated as:

```
sf = 0.91 + 0.01 * Σ(scale factor ratings)
```

The `B` value (which is `sf` in the nominal person-months formula) can range from 0.91 to 1.23. Each scale driver has six rating levels (e.g., very low, low, nominal, high, very high, extra high), with corresponding numerical values.

Example of Scale Factor Calculation:
If a project has the following scale driver ratings:
* PREC: Very high (1.24)
* FLEX: Very low (5.07)
* RESL: High (2.83)
* TEAM: High (2.19)
* PMAT: Low (6.24)

Then, `sf = 0.91 + 0.01 * (1.24 + 5.07 + 2.83 + 2.19 + 6.24) = 0.91 + 0.01 * 17.57 = 0.91 + 0.1757 = 1.0857`.
For a 10 KSLOC project, the effort would be
`2.94 * 10^1.0857 = 35.8 person-months`.

6. Reuse Model

The reuse model accounts for the effort associated with incorporating existing code. It considers:
* Overhead for assessing, selecting, and assimilating components.
* The fact that even small modifications can generate disproportionately large costs.

There are two versions:
* **Black-box reuse**: Code is reused without modification. An effort estimate (PM) is computed.
* **White-box reuse**: Code is modified. A size estimate equivalent to the number of lines of new source code is computed, which then adjusts the size estimate for the new code.

Reuse Estimates Formulas:
1. **For generated (reused) code**:

```
PM = (ASLOC * AT / 100) / ATPROD
```

Where:
* `ASLOC` = Number of lines of generated code
* `AT` = Percentage of code automatically generated
* `ATPROD` = Productivity of engineers in integrating this code

1. **When code has to be understood and integrated**:

```
ESLOC = ASLOC * (1 - AT / 100) * AAM
```

Where:
* `ESLOC` = Equivalent Source Lines of Code
* `ASLOC` and `AT` are as above.
* `AAM` = Adaptation Adjustment Multiplier, computed from costs of changing reused code, understanding integration, and reuse decision-making.

7. Post-Architecture Model and Effort Multipliers

The Post-Architecture model uses the same fundamental formula as the Early Design Model but with a more extensive set of 17 associated multipliers (compared to 7 in the Early Design Model).

- **Code Size Estimation**:
- Number of lines of new code to be developed.
- Equivalent number of lines of new code (computed using the reuse model).
- Estimate of lines of code to be modified due to requirements changes.

Effort Multipliers: These factors further refine the effort estimate based on specific project characteristics. Examples include:
* `RCPX`: Product reliability and complexity
* `RUSE`: Reuse required
* `PDIF`: Platform difficulty
* `PERS`: Personnel capability
* `FCIL`: Facilities available
* `SCED`: Schedule pressure

These multipliers have values typically rated on a 7-point scale (e.g., extra low to extra high).

Example of Effort Multiplier Application:
If an initial estimate is 35.8 person-months, and specific effort multipliers are:
* RCPX (very high): 1.91
* PDIF (very high): 1.81
* PERS (extra high): 0.50
* PREX (nominal): 1.00
(All other factors nominal, i.e., 1.00)

The revised estimate becomes `35.8 * 1.91 * 1.81 * 0.50 * 1.00 = 61.9 person-months`.

8. COCOMO II Scaling Exponent Approach

The nominal person-months can also be calculated using a slightly different form of the core equation:

```
Nominal person-months = A * (size)^B
```

Where `B` is the scaling exponent, calculated as:

```
B = 0.91 + 0.01 * Σ(scale factor ratings)
```

This `B` value ranges from 0.91 to 1.23, reflecting the impact of the five scale factors.

Example:
For a project with all nominal cost drivers and scale drivers, the Effort Adjustment Factor (EAF) is 1.0, and the exponent `E` (which is `B`) is 1.0997. If the project is 8,000 source lines of code (8 KSLOC):

```
Effort = 2.94 * (1.0) * (8)^1.0997 = 28.9 Person-Months
```

This demonstrates how COCOMO II provides a structured way to estimate effort by incorporating various project attributes.

Key Takeaways

- [ ] COCOMO II is an updated estimation model addressing modern software development complexities like reuse, GUIs, and new paradigms.
- [ ] The model evolved from COCOMO 81 to provide more accurate cost and schedule estimations for contemporary projects.

- [ ] COCOMO II includes four sub-models: Application Composition, Early Design, Reuse, and Post-Architecture, each applicable at different project stages.
- [ ] SLOC (Source Lines of Code) is the primary size metric in COCOMO II, offering a more logical count than DSI.
- [ ] Application points, based on screens, reports, and modules, are crucial for estimating effort in GUI-intensive and prototyping projects.
- [ ] Scale drivers (Precedentedness, Flexibility, Architecture/Risk Resolution, Team Cohesion, Process Maturity) determine the exponent in the effort equation, significantly impacting project duration and cost.
- [ ] Effort multipliers further refine estimates by accounting for specific product, platform, personnel, and project characteristics.
- [ ] The Reuse Model is vital for calculating the effort involved in integrating existing or automatically generated code, distinguishing between black-box and white-box reuse.
- [ ] The Post-Architecture Model provides the most detailed and accurate estimates, utilizing a comprehensive set of 17 multipliers.

Conclusion

COCOMO II stands as a testament to the continuous evolution required in software engineering practices. By moving beyond the limitations of its predecessor, it offers a robust framework for estimating effort and cost in an era defined by component reuse, intricate graphical interfaces, and agile development methodologies. Its multi-model approach allows for progressive refinement of estimates as a project matures, from initial concept to detailed architecture.

The integration of scale drivers and effort multipliers provides a nuanced understanding of how various project attributes—from team dynamics and process maturity to product complexity and platform difficulty—influence the overall development effort. For students and professionals alike, a thorough grasp of COCOMO II is indispensable for effective project planning, resource allocation, and risk management, ultimately contributing to more predictable and successful software delivery in a rapidly changing technological landscape.